

توثيق تقني شامل

منصة توظيف إلكترونية (Job Board Platform)

مشروع تدريبي لبناء REST API احترافي باستخدام

ASP.NET Core 8 · Clean Architecture · CQRS + MediatR · EF Core · JWT

إعداد: عبدالرحمن سعيد غريب — Backend Developer

مستند تخطيط قبل بدء التنفيذ — إصدار 1.0

1) نظرة عامة على المشروع

بمشروع واحد بعينه، وكمان يفتحلك مجال تضيف عليه لاحقاً خطوة بسيطة في اتجاه الـ AI/ML من خلال خاصية الـ Matching Score. ومتابعة حالة طلباتهم. الهدف الأساسي من المشروع تدريبي بالكامل: تطبيق نفس المعمارية والأنماط اللي اتدربت عليها في مشروع JWT. عبارة عن منصة توظيف إلكترونية (Job Board) تسمح للشركات بنشر الوظائف الشاغرة، وللباحثين عن عمل بالتصفح والتقديم عليها المشروع

1.1 الأهداف التعليمية من المشروع

- ترسيخ فصل الطبقات (Domain / Application / Infrastructure / API) وعدم تسريب تفاصيل EF Core لطبقة الـ Application
- التمرين على CQRS باستخدام MediatR مع فصل حقيقي بين الـ Commands و الـ Queries
- بناء نظام صلاحيات متعدد الأدوار (Role-based + Policy-based Authorization)
- التعامل مع رفع الملفات (CV Upload) وربطها بالـ Domain Model
- تصميم Pagination / Filtering / Sorting بشكل Generic قابل لإعادة الاستخدام
- كتابة Validation احترافي باستخدام FluentValidation مع Pipeline Behavior
- توحيد شكل الأخطاء عبر Global Exception Middleware بصيغة ProblemDetails
- خطوة تمهيدية بسيطة نحو الـ AI/ML عبر خوارزمية Matching Score قائمة على القواعد

1.2 المكسب التقني (Tech Stack)

الطبقة / الغرض	التقنية المقترحة
إطار العمل	.NET 8 / ASP.NET Core Web API
قاعدة البيانات	SQL Server + Entity Framework Core (Code-First)
المصادقة	ASP.NET Identity + JWT (Access + Refresh Token)
التحقق من صحة البيانات	FluentValidation
تحويل الكائنات	AutoMapper
الوسيط (Mediator)	MediatR (Commands / Queries / Notifications)
التسجيل (Logging)	Serilog
المهام المجدولة	Hangfire أو BackgroundService
التوثيق	Swagger / OpenAPI
الاختبار	xUnit + Moq + FluentAssertions

2) معمارية المشروع — Clean Architecture

نفس التقسيم اللي استخدمته في Badil، بس هنا هيبقى التركيز على تنظيف الحدود بين الطبقات من أول commit، بدل ما تصلحها بعدين.

الطبقة	المسؤولية	أهم المحتويات
Domain	مفيهش أي Dependency على أي طبقة ثانية قلب النظام،	Entities, Enums, Value Objects, Domain Exceptions, Interfaces أساسية (IRepository<T>)
Application	الحالات الاستخدامية (Use Cases) وتنسيقها منطق	Commands, Queries, Handlers, DTOs, Validators, Mapping Profiles, Pipeline Behaviors, Interfaces (IEmailService, IFileService...)
Infrastructure	الفعلي لل Interfaces والتعامل مع العالم الخارجي التنفيذ	DbContext, Entity Configurations, Migrations, Repository/UnitOfWork, JwtService, FileService, EmailService
API	نقطة الدخول (Composition Root)	Controllers لل Mediator (Command/Query رقيقة جدا، بترسل (بس), Middleware, DI Registration, Swagger

2.1 قاعدة أساسية لل Dependency Rule

الطريقة العملية للتأكد من كده إنك متسمحش لأي using ل Microsoft.EntityFrameworkCore يظهر جوه مجلد Application. يعتمد على Application و Domain، لكن Domain و Application منعزلين تماما عن أي تفاصيل تقنية زي EF Core أو ASP.NET. الاتجاه الوحيد المسموح لل Dependencies هو من الخارج للداخل: API يعتمد على Application، و Infrastructure

(3) الكيانات (Domain Entities)

تصميم الكيانات الأساسية التي هيتبني عليها كل حاجة، مع أنواع البيانات والقيود.

المستخدم — User (ApplicationUser : IdentityUser)

يمثل أي مستخدم في النظام، وييرث من IdentityUser عشان يستفيد من ASP.NET Identity جاهز.

الحقل	النوع	القيود	الوصف
FullName	string	Required, MaxLength(100)	الاسم الكامل للمستخدم
ProfilePictureUrl	string?	Nullable	رابط الصورة الشخصية
Role	enum (Admin/ Employer/JobSeeker)	Required	دور المستخدم في النظام
IsActive	bool	Default: true	هل الحساب مفعّل أم موقوف من الأدمن
CreatedAt	DateTime	Default:.UtcNow	تاريخ إنشاء الحساب

الشركة — Company

بيانات الشركة صاحبة الوظائف، مرتبطة بواحد أو أكثر من مستخدمي Employer.

الحقل	النوع	القيود	الوصف
Id	Guid	PK	المعرف الفريد
Name	string	Required, MaxLength(150)	اسم الشركة
Description	string	Required	نبذة عن الشركة
LogoUrl	string?	Nullable	رابط اللوجو
Website	string?	Nullable	الموقع الإلكتروني
Industry	string	Required	المجال الصناعي
OwnerUserId	string (FK)	Required	صاحب الشركة (Employer)
IsVerified	bool	Default: false	موافقة الأدمن على الشركة

الوظيفة — Job

الإعلان الوظيفي المنشور من قبل شركة معينة.

الحقل	النوع	القيود	الوصف
Id	Guid	PK	المعرف الفريد

الحقل	النوع	القيود	الوصف
Title	string	Required, MaxLength(120)	المسمى الوظيفي
Description	string	Required	الوصف التفصيلي للوظيفة
Requirements	string	Required	المتطلبات والمؤهلات
Location	string	Required	موقع العمل
WorkType	enum (FullTime/PartTime/Remote/Internship)	Required	نوع الدوام
MinSalary / MaxSalary	decimal?	Nullable	نطاق الراتب (اختياري)
Status	enum (Open/Closed/Draft)	Default: Open	حالة الوظيفة
Deadline	DateTime?	Nullable	آخر موعد للتقديم
CompanyId	Guid (FK)	Required	الشركة الناشرة
ViewsCount	int	Default: 0	عدد المشاهدات (للتحليلات)
CreatedAt / UpdatedAt	DateTime	Required	تواريخ الإنشاء والتعديل

التصنيف والمهارة — JobCategory / Skill

Many-to-One مع الوظيفة (تصنيف واحد لكل وظيفة)، أما المهارة فعلاقتها Many-to-Many مع الوظيفة عبر جدول وسيط JobSkill. التصنيف نوع علاقته

الحقل	النوع	القيود	الوصف
Id	Guid	PK	المعرف الفريد (لكل من الكيانين)
Name	string	Required, Unique	اسم التصنيف أو المهارة
JobId (في JobSkill)	Guid (FK)	Composite PK	ربط الوظيفة بالمهارة
SkillId (في JobSkill)	Guid (FK)	Composite PK	ربط المهارة بالوظيفة

الطلب — Application

طلب تقديم مستخدم JobSeeker على وظيفة معينة، وهو قلب ال Workflow في المشروع.

الحقل	النوع	القيود	الوصف
Id	Guid	PK	المعرف الفريد
JobId	Guid (FK)	Required	الوظيفة المتقدم لها

الحقل	النوع	القيود	الوصف
JobSeekerId	string (FK)	Required	المستخدم المتقدم
ResumeId	Guid (FK)	Required	السيرة الذاتية المرفقة
CoverLetter	string?	Nullable, MaxLength(1000)	خطاب تقديم اختياري
Status	enum (Applied/Reviewed/Interview/Accepted/Rejected)	Default: Applied	حالة الطلب
AppliedAt	DateTime	Default: UtcNow	تاريخ التقديم
StatusUpdatedAt	DateTime?	Nullable	آخر تحديث للحالة
Unique Constraint	—	(JobId, JobSeekerId)	منع التقديم على نفس الوظيفة مرتين

السيرة الذاتية — Resume

علاقة One-to-One مع JobSeeker، تحتوي بيانات مهيكلة بالإضافة للملف المرفوع.

الحقل	النوع	القيود	الوصف
Id	Guid	PK	المعرف الفريد
JobSeekerId	string (FK)	Required, Unique	صاحب السيرة الذاتية
FileUrl	string	Required	رابط ملف الـ PDF
Summary	string?	Nullable	نبذة مختصرة
YearsOfExperience	int	Default: 0	عدد سنوات الخبرة
Skills	ICollection<Skill>	Many-to-Many	المهارات المرتبطة بالسيرة (لحساب Matching Score)

أخرى — SavedJob / Notification / RefreshToken

كيانات مساعدة تدعم تجربة الاستخدام والأمان.

الحقل	النوع	القيود	الوصف
SavedJob.JobSeekerId + JobId	Composite Key	Unique	الوظائف المحفوظة في المفضلة
Notification.UserId	string (FK)	Required	صاحب الإشعار
Notification.Message / IsRead	string / bool	Required / Default:false	محتوى الإشعار وحالة القراءة
RefreshToken.Token / ExpiresAt / IsRevoked	string / DateTime / bool	Required	لإدارة تجديد الجلسات بأمان

4) العلاقات بين الكيانات

الكيان الأول	نوع العلاقة	الكيان الثاني	ملاحظة
User (Employer)	One-to-Many	Company	تتبع لأكثر من Employer لاحقا (Team) — بس نبداً بواحد الشركة الواحدة تقدر
Company	One-to-Many	Job	الشركة الواحدة عندها أكثر من وظيفة
Job	Many-to-One	JobCategory	كل وظيفة تصنيف واحد بس
Job	Many-to-Many	Skill	عبر جدول وسيط JobSkill
User (JobSeeker)	One-to-One	Resume	كل باحث عمل سيرة ذاتية واحدة فعالة
Resume	Many-to-Many	Skill	لحساب نسبة التطابق مع الوظائف
User (JobSeeker)	One-to-Many	Application	المستخدم يقدر يقدم على أكثر من وظيفة
Job	One-to-Many	Application	الوظيفة تستقبل أكثر من طلب
User	One-to-Many	Notification	كل مستخدم له سجل إشعارات
User (JobSeeker)	Many-to-Many	Job (SavedJob عبر)	قائمة الوظائف المفضلة

برسم ال ERD ده على ورقة أو أداة زي dbdiagram.io قبل ما تكتب أي Migration، بالظبط زي ما عملت في مستند ال E-commerce API قبل كده. نصيحة عملية: ابدأ

5) مصفوفة الأدوار والصلاحيات

الميزة	Admin	Employer	JobSeeker
إدارة المستخدمين (تفعيل/تعطيل)	☐	☐	☐
إدارة التصنيفات والمهارات	☐	☐	☐
توثيق حسابات الشركات (Verify)	☐	☐	☐
إنشاء / تعديل بيانات الشركة	☐	☐ (شركته فقط)	☐
نشر / تعديل / حذف وظيفة	☐	☐ (وظائفه فقط)	☐
عرض المتقدمين على وظيفة	☐	☐ (وظائفه فقط)	☐
تغيير حالة طلب التقديم	☐	☐ (على وظائفه)	☐
تصفح والتقديم على الوظائف	☐	☐	☐
رفع / تعديل السيرة الذاتية	☐	☐	☐
حفظ وظيفة في المفضلة	☐	☐	☐
لوحة إحصائيات عامة (Dashboard)	☐	☐	☐

للتحقق إن ال Employer فعلا هو صاحب الوظيفة أو الشركة التي يحاول تعديلها — ده اللي ييمنع مستخدم من تعديل بيانات مش بتاعته. التنفيذ في الكود ييبقى بمزيج من ["Authorize(Roles="Employer")] للتحقق من الدور، و Policy/Handler مخصص (زي IAuthorizationHandler)

6 المتطلبات الوظيفية التفصيلية

6.1 Admin

- تسجيل دخول منفصل بصلاحيات Admin (يتميز في الداتايز عبر Seeding مش عن طريق تسجيل عادي)
- عرض قائمة كل المستخدمين مع فترة حسب الدور والحالة، وتفعيل/تعطيل أي حساب
- Skill و JobCategory كامل على CRUD
- مراجعة وتوثيق حسابات الشركات الجديدة (Verify/Reject) قبل ما تقدر تنشر وظائف
- لوحة إحصائيات: إجمالي الوظائف، إجمالي الطلبات، أكثر 5 شركات نشاطاً، أكثر 5 تصنيفات طلباً

6.2 Employer

- إنشاء بروفایل شركة (مرة واحدة لكل Employer في النسخة الأولى) وتعديل بياناته
- نشر وظيفة جديدة بكل تفاصيلها (العنوان، الوصف، المتطلبات، الراتب، نوع الدوام، الديدلاين، المهارات المطلوبة)
- تعديل أو إغلاق وظيفة يدوياً، وإغلاقها تلقائياً لما يعدي ال Deadline (عبر Background Job)
- عرض قائمة كل المتقدمين على وظيفة معينة، مع فترة حسب Status أو ترتيب حسب نسبة التطابق
- عرض تفاصيل كل طلب تقديم (بيانات المتقدم + السيرة الذاتية + خطاب التقديم)
- تغيير حالة الطلب عبر مسار منطقي محدد: Applied → Reviewed → Interview → Accepted/Rejected
- استقبال إشعار داخلي (وربما إيميل) لما حد يقدم على وظيفة من وظائفه

6.3 JobSeeker

- إنشاء وتعديل البروفايل الشخصي، ورفع سيرة ذاتية (ملف PDF) مرة واحدة قابلة للتحديث
- إضافة المهارات وسنوات الخبرة والملخص الشخصي كبيانات مهيكلة (مش بس ملف)
- تصفح الوظائف المفتوحة مع فترة (تصنيف، موقع، نوع الدوام، نطاق راتب) وترتيب و Pagination
- التقديم على وظيفة واحدة مرة واحدة فقط (مع رسالة خطأ واضحة عند محاولة التكرار)
- متابعة كل طلباته وحالاتها في صفحة واحدة (My Applications)
- حفظ وظيفة في المفضلة (Wishlist) وإزالتها لاحقاً
- استقبال إشعار لما تتغير حالة أي طلب من طلباته

7) تصميم CQRS — أمثلة تفصيلية

بيه. ده بيخليك تقدر تضيف Behaviors (زي Logging أو Validation) على كل ال Pipeline من غير ما تلمس منطق كل Handler. بدل ما تعمل Service Layer تقليدي، كل Use Case بيتحول ل Command أو Query منفصل، له Handler خاص

7.1 موديول الوظائف (Jobs)

- CreateJobCommand □ CreateJobCommandHandler
- (بيتحقق من الملكية أولاً) UpdateJobCommand □ UpdateJobCommandHandler
- DeleteJobCommand (Soft Delete) □ DeleteJobCommandHandler
- CloseExpiredJobsCommand □ Background Job بيتنفذ من
- (ViewsCount بيزود) GetJobByIdQuery □ GetJobByIdQueryHandler
- GetJobsListQuery (Filter + Sort + Pagination) □ GetJobsListQueryHandler

7.2 موديول الطلبات (Applications)

- Open بيتحقق من عدم التكرار + من إن الوظيفة لسه □ ApplyToJobCommand
- بيتحقق إن اللي بيغير الحالة هو صاحب الوظيفة □ UpdateApplicationStatusCommand
- JobSeeker □ GetMyApplicationsQuery
- Employer □ GetApplicationsForJobQuery (Filter + Sort by MatchScore)

7.3 موديول المصادقة (Auth)

- (Employer/JobSeeker) نفسه Request بيحدد الدور من ال □ RegisterCommand
- Access Token + Refresh Token بيرجع □ LoginCommand
- Refresh Token بيتحقق من صلاحية ال □ RefreshTokenCommand
- لتسجيل الخروج بأمان □ RevokeTokenCommand

7.4 المقترحة Pipeline Behaviors

- Handler قبل ما يوصل ل FluentValidation Validators بينفذ كل ال — ValidationBehavior
- الوقت المستغرق في التنفيذ Request يسجل اسم ال — LoggingBehavior
- ييمسك أي استثناء غير متوقع ويحواله لصيغة موحدة — UnhandledExceptionBehavior

8) ال API Endpoints الكاملة

8.1 Auth

الصلاحية	الوصف	Endpoint	Method
عام	تسجيل حساب جديد (Employer/JobSeeker)	/api/auth/register	POST
عام	تسجيل الدخول وإصدار التوكنز	/api/auth/login	POST
عام (ب Refresh Token صالح)	تجديد ال Access Token	/api/auth/refresh-token	POST
مستخدم مسجل	تسجيل الخروج / إلغاء التوكن	/api/auth/revoke-token	POST

8.2 Companies

الصلاحية	الوصف	Endpoint	Method
Employer	إنشاء بروفایل الشركة	/api/companies	POST
Employer — فقط المالك	تعديل بيانات الشركة	/api/companies/{id}	PUT
عام	عرض تفاصيل شركة ووظائفها	/api/companies/{id}	GET
Admin	توثيق الشركة	/api/companies/{id}/verify	PATCH

8.3 Jobs

الصلاحية	الوصف	Endpoint	Method
عام	قائمة الوظائف (فلتر+ترتيب+صفحات)	/api/jobs	GET
عام	تفاصيل وظيفة	/api/jobs/{id}	GET
Employer	نشر وظيفة جديدة	/api/jobs	POST
Employer — فقط المالك	تعديل وظيفة	/api/jobs/{id}	PUT
Employer — فقط المالك	حذف (Soft Delete) وظيفة	/api/jobs/{id}	DELETE
Employer — فقط المالك	إغلاق وظيفة يدويا	/api/jobs/{id}/close	PATCH

8.4 Applications

الصلاحية	الوصف	Endpoint	Method
JobSeeker	التقديم على وظيفة	/api/applications	POST
JobSeeker	طلباتي وحالاتها	/api/applications/my	GET

الصلاحية	الوصف	Endpoint	Method
Employer — فقط المالك	متقدمين وظيفة معينة	/api/jobs/{jobId}/applications	GET
Employer — فقط المالك	تغيير حالة الطلب	/api/applications/{id}/status	PATCH

8.5 Resumes / SavedJobs / Admin

الصلاحية	الوصف	Endpoint	Method
JobSeeker	رفع/تحديث السيرة الذاتية	/api/resumes	POST
JobSeeker	عرض سيرتي الذاتية	/api/resumes/my	GET
JobSeeker	إضافة وظيفة للمفضلة	/api/saved-jobs/{jobId}	POST
JobSeeker	إزالة من المفضلة	/api/saved-jobs/{jobId}	DELETE
Admin	إحصائيات عامة	/api/admin/dashboard	GET
Admin	تفعيل/تعطيل مستخدم	/api/admin/users/{id}/toggle-status	PATCH

(زي /api/jobs/) لازم تدعم query params موحدة: pageNumber, pageSize, sortBy, sortDirection. بالإضافة لفلاتر خاصة بكل Endpoint. كل ال Endpoints اللي بترجع قوائم

9) قواعد ال Validation الأساسية

أهم قواعد التحقق	الأمر (Command)
— مطلوبين Requirements و Description — MaxLength(120) مطلوب و Title MinSalary <= MaxSalary — لو موجود لازم يكون في المستقبل Deadline	CreateJobCommand
أن يكون Role — يحقق الحد الأدنى للتعقيد Password — صيغة صحيحة وغير مكرر Email فقط JobSeeker أو Employer يجب	RegisterCommand
وحالتها Open — المستخدم عنده Resume مرفوعة — عدم وجود طلب سابق لنفس الوظيفة الوظيفة موجودة	ApplyToJobCommand
الانتقال بين الحالات لازم يتبع تسلسل منطقي (مينفعش من Applied ل Accepted مباشرة)	UpdateApplicationStatusCommand
صحيحة URL لو موجود) صيغة Website — مطلوب Description — مطلوب Name	CreateCompanyCommand

10) الأمان والمصادقة

10.1 آلية العمل

- عند تسجيل الدخول، السيرفر يصدر Access Token (صالح 15-30 دقيقة) و Refresh Token (صالح أيام أطول).
- ال Access Token يمتلكن فى ال Claims: UserId, Role, Email
- لما ال Access Token ينتهى، الفرونت إند يبعث ال Refresh Token على /refresh-token عشان ياخذ زوج جديد.
- Refresh Token يمتلكن فى قاعدة البيانات (مش بس فى التوكن) عشان تقدر تلغيه (Revoke) عند تسجيل الخروج أو الشك فى اختراق ال Token
- زى MustBeJobOwner بتتحقق إن ال CompanyId أو ال EmployerId فى ال Claims يبطابق صاحب المورد المطلوب تعديله
- استخدام Policy مخصصة

10.2 مثال على بنية ال JWT Claims

sub: userId | role: Employer | email: user@mail.com | exp: 1750000000

11) اعتبارات تصميم قاعدة البيانات

- DbContext في ال IsDeleted + Global Query Filter عبر عمود Application و Job على Soft Delete
 - لتسريع البحث والفلتر Job في جدول (Title, Location) مركب على Index
 - لمنع التقديم المكرر على مستوى الداتايز مش الكود بس Application في جدول (JobId, JobSeekerId) على Unique Index
 - استخدام RowVersion (Concurrency Token) في جدول Job لتفادي تعارض التعديلات المتزامنة من أكثر من Employer
 - الإعدادات في <T> IEntityConfiguration; منفصلة لكل Entity وتحميلها بـ ApplyConfigurationsFromAssembly ()
- فصل
- لأن حذف وظيفة مينفعش يمسح تاريخ الطلبات، Restrict لازم — Application و Job ممنوع بين Cascade Delete

(12) معالجة الأخطاء الموحدة

خطأ ييرجع بصيغة ProblemDetails قياسية عبر Global Exception Middleware، بدل ما كل Controller يعمل try/catch بنفسه. كل

نوع الاستثناء	HTTP Status	مثال حالة
ValidationException	400 Bad Request	بيانات إنشاء وظيفة ناقصة أو غير صحيحة
NotFoundException	404 Not Found	محاولة الوصول لوظيفة بمعرف غير موجود
ForbiddenAccessException	403 Forbidden	ييحاول يعدل وظيفة مش بتاعته Employer
UnauthorizedAccessException	401 Unauthorized	توكن منتهي الصلاحية أو غير موجود
ConflictException	409 Conflict	محاولة تقديم مكرر على نفس الوظيفة
استثناء غير متوقع	500 Internal Server Error	أي خطأ غير معالج، بيتسجل في Serilog

(13) متطلبات غير وظيفية

- Pagination Generic: PagedResult<T> يحتوي Items, TotalCount, PageNumber, PageSize
- Handler متداخلة في ال If/Else بدل Specification Pattern قابل للتوسع عبر Filtering
- لقائمة الوظائف الرئيسية لتقليل الضغط على الداتايز (In-Memory) بسيط Caching
- لمنع إغراق النظام بطلبات وهمية (ApplyToJob) التقديم Endpoint على Rate Limiting
- Logs عشان تقدر تتبع رحلة الطلب في ال Request لكل Correlation Id مع Serilog منظم عبر Logging
- لإغلاق الوظائف الي عدى ديدلاينها تلقائيا (Hangfire) يومي Background Job

14) الميزات الإضافية (Bonus)

14.1 AI/ML خطوة تمهيدية للـ Matching Score

أو حتى تجربة نموذج Embeddings بسيط لمقارنة النصوص بدل التطابق الحرفي للمهارات — وده أول خطوة عملية ليك ناحية الـ NLP. مع مهارات الـ Job المطلوبة $(\text{intersection} / \text{required skills count} \times 100)$ ، ولاحقاً تقدر تطورها بإضافة وزن لسنوات الخبرة بسيطة قائمة على القواعد (Rule-Based) قبل ما تفكر في أي Machine Learning حقيقي: بتحسب نسبة تقاطع مهارات الـ Resume خوارزمية

14.2 ميزات أخرى مقترحة

- عند تغيير حالة الطلب عبر Background Service Email Notifications
- على عنوان ووصف الوظيفة SQL Server Full-Text Index باستخدام Full-text Search
- عدد المشاهدات مقابل عدد التقديمات لكل وظيفة: Employer Analytics Dashboard
- فقط Polling بدل الاعتماد على (Real-time) لإشعارات لحظية SignalR

15) خطة التنفيذ التفصيلية (7 أيام)

الأساس (Domain + Application) — اليوم 1

- تصميم كل ال Entities وال Enums في طبقة Domain
- تعريف ال Interfaces الأساسية (IRepository, IUnitOfWork, IJwtService, IFileService)
- إنشاء ال DTOs الأولية و Mapping Profiles

قاعدة البيانات (Infrastructure) — اليوم 2

- إعداد ال DbContext وال Entity Configurations لكل Entity
- أول Migration وتجربة الاتصال بقاعدة البيانات
- تطبيق Repository Pattern + UnitOfWork

المصادقة والصلاحيات — اليوم 3

- دمج ASP.NET Identity وإعداد ال Roles (Seeding)
- بناء ال JwtService (إصدار Access + Refresh Token)
- Endpoints الخاصة بـ Register / Login / RefreshToken

موديول الوظائف (Jobs) — اليوم 4

- Commands: Create / Update / Delete / Close
- Queries: GetById / GetList مع Filtering + Sorting + Pagination
- التحقق من ملكية الوظيفة Policy

الطلبات ورفع الملفات — اليوم 5

- بناء ال FileService لرفع السير الذاتية (PDF) بشكل آمن
- (مع كل التحققات) (تكرار، حالة الوظيفة ApplyToJob Command)
- Endpoints لـ JobSeeker و Employer متابعة الطلبات

الإشعارات ومعالجة الأخطاء — اليوم 6

- Global Exception Middleware + ProblemDetails
- بسيط عند تغيير حالة الطلب Notification Service
- تفعيل FluentValidation عبر Pipeline Behavior

الاختبار والتلميع — اليوم 7

- اختبار كل ال Endpoints عبر Swagger/Postman بسيناريوهات مختلفة
- مراجعة ال Response الموحد للأخطاء والنجاح
- كتابة ال README شامل وتوثيق ال Endpoints

(16) معايير القبول (Definition of Done)

- كل ال Endpoints موثقة في Swagger وتشغل بدون أخطاء 500 غير متوقعة •
- لا يوجد أي using ل EF Core داخل طبقة Application •
- كل Command له Validator خاص به، ومفیش تحقق من صحة البيانات جوه ال Controller •
- الصلاحيات مطبقة فعليا (تجربة يدوية: محاولة تعديل وظيفة مش بتاعتك لازم ترجع 403) •
- الأخطاء كلها بترجع بصيغة موحدة (ProblemDetails) مش Stack Trace خام •
- TotalCount شغالة على كل القوائم مع أرقام صحيحة ل Pagination •

- Matching القائم على القواعد بنموذج Embeddings حقيقي (زي Sentence Transformers) لمقارنة نص ال CV بنص الوظيفة • استبدال Score
- إضافة Recommendation Engine بسيط يقترح وظائف لل JobSeeker بناء على تاريخ تصفحه وتقديماته •
- تحويل جزء من النظام لـ Microservices لو حبيت تتمرن على الموضوع لاحقا (خدمة منفصلة للإشعارات مثلا) •
- دمج خدمة OCR/NLP لاستخراج المهارات تلقائيا من ملف ال CV المرفوع بدل إدخالها يدويا •

المستند ده نقطة انطلاق للتخطيط قبل الكود، مش عقد ملزم — لو لقيت أثناء التنفيذ إن حاجة محتاجة تبسط أو توسع، عدلها براحتك واستمر.